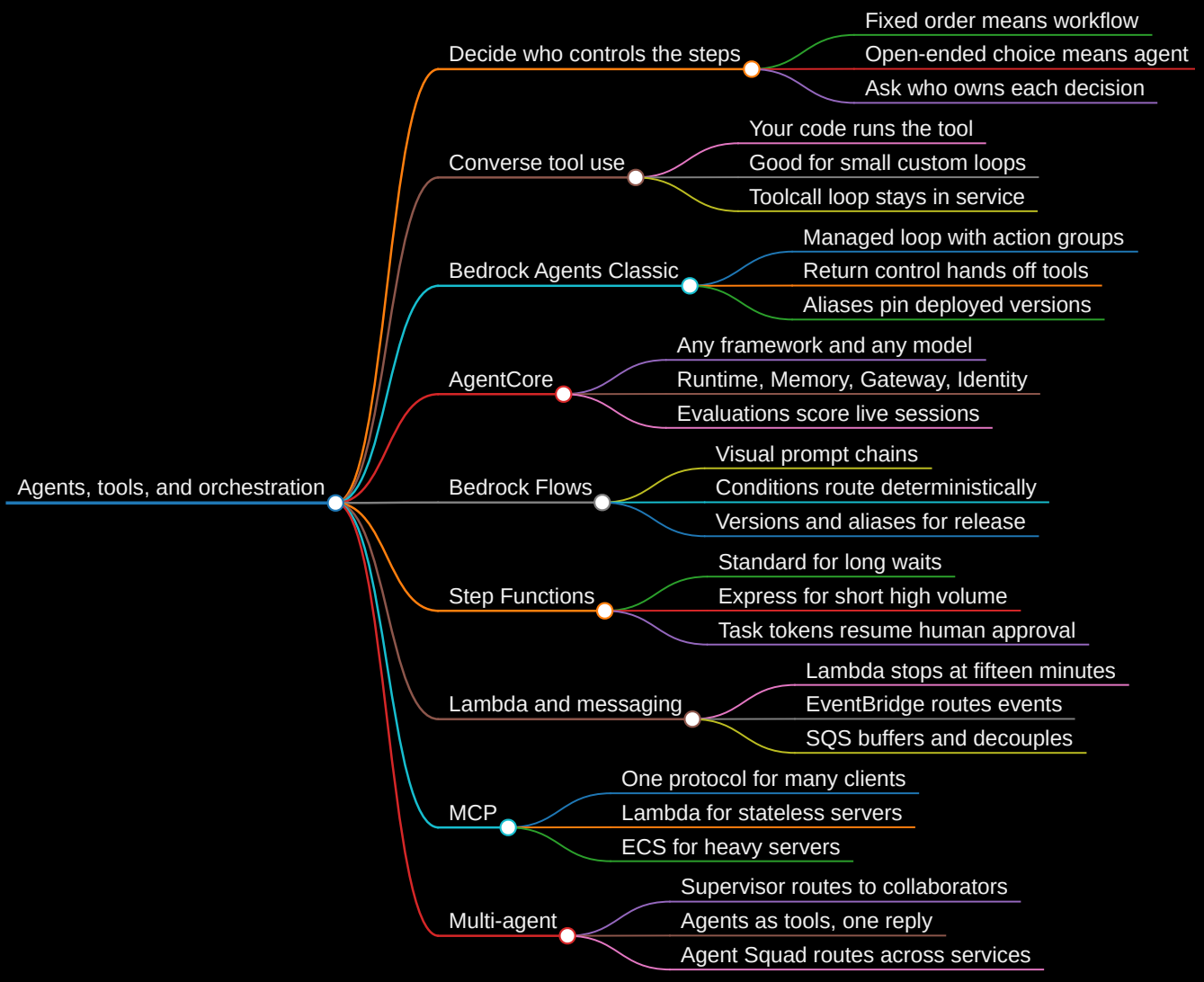
Module 4: Agents, tools, and orchestration

After this module, you can read a long agent or workflow question and find the one constraint that decides the design. You can separate three kinds of control: your code runs a tool, a managed agent loop picks tools for you, or a workflow you draw fixes the order of steps. You can also reject each wrong option with one reason, even when the option names a real AWS service.



The deciding constraint

Start from the scenario and ask one question first: who decides the next step, the model or your code? The answer removes most of the options before any

service name matters. A model that picks the next step needs an agent loop, guardrails on tool use, and a trace. Code that picks the next step needs a workflow, a flow, or a plain function.

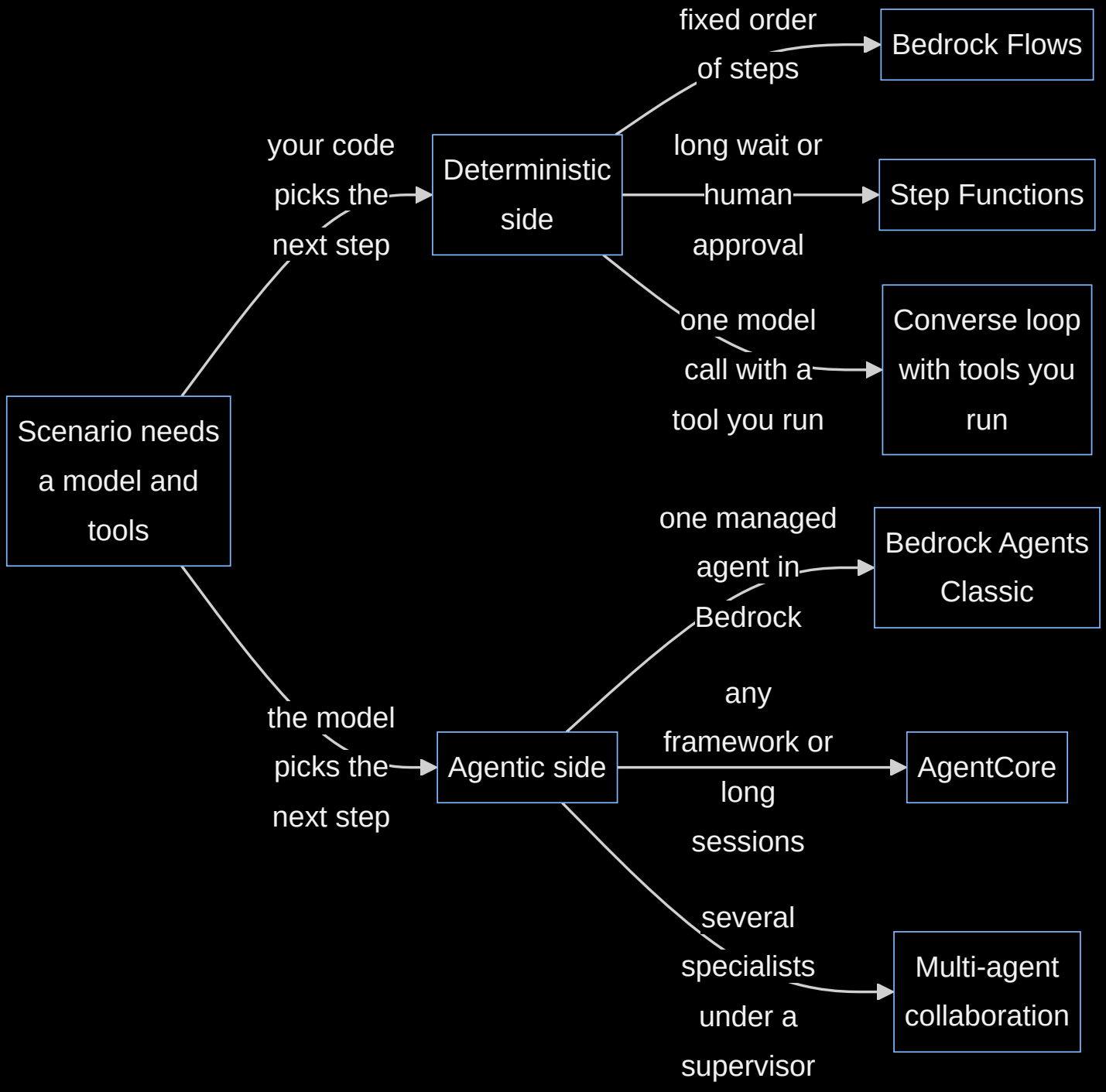


Figure: The first split is who picks the next step, then the wait and framework needs pick the service.

Use these signal words to move down each branch. Phrases such as "in this exact order", "every time", and "for audit" point to a workflow. Phrases such as "decide which tool to use", "any framework", and "remember across visits" point to an agent. Phrases such as "wait for a person", "several days", and "approval link" point to a callback in Step Functions.

Rule: If you can draw every step before the request arrives, do not choose an agent.

Exam tip: Read the verb in the question. "Decide" and "choose" point to a model. "Run", "route", and "wait" usually point to a workflow or a function.

Converse tool use: your code runs the tool

The Converse API gives one request shape for models that support it. A request can include a tool configuration with tool definitions and a tool choice setting. When the model asks for a tool, the model reply contains the tool request, and your application runs the function and sends the result back in the next message.

The loop belongs to your code in this pattern. Amazon Bedrock does not call your Lambda function for you. You decide how many turns the loop can take, how errors are retried, and where the state is saved. This control is the reason to choose it.

Converse tool use fits a small custom loop that already lives in your service. It also fits a team that wants to keep one request format across models. Lambda, a container, or an AgentCore Runtime endpoint can host the loop. The model choice does not change the loop owner.

Rule: Choose Converse tool use when your own code must own the tool loop, the retries, and the state.

Trap: Choosing a Bedrock agent because the question mentions tools. Converse does not run tools for you, so the managed agent loop adds a service you do not need.

Converse pairs with Lambda for the tool body, with Step Functions when the loop must survive long waits, and with guardrails on the request. Limits for the number of tools or turns are Not in source for this module.

Source: Sundog course extract, slide page 21 (Converse request fields including toolConfig) and slide page 20 (bedrock-runtime Converse). Converse request reference: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how-it-works.html was attempted and returned only a title, so no fact here comes from that page.

Bedrock Agents Classic: the managed agent loop

Amazon Bedrock Agents is a managed loop. The foundation model reads the user message, plans steps, picks an action, fills in its parameters, and writes the final answer. The agent calls action groups, and it can also search attached knowledge bases.

AWS states that Amazon Bedrock Agents Classic is no longer open to new customers and is in maintenance mode. AWS points new builds to Amazon Bedrock AgentCore. Existing customers can keep using it as normal.

The loop is managed, so you give up control of the steps the model takes. You control the instructions, the action definitions, the executor for each action, and the alias that clients call. The model still decides when to call each action. This is why the descriptions matter so much.

The Sundog course describes the same idea in plain terms. The action group defines a tool, the prompt tells the model when to use it, and the description is used to pull the needed values from the user message. The course also says the model can ask the user for missing values before a call.

Rule: For a new build, check AgentCore first. Keep Bedrock Agents Classic for existing agents that already run well.

Trap: Treating the action description as optional text. The model uses it to choose the action, so a vague description causes wrong calls.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-action-create.html (maintenance and action group summary). Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html (maintenance note). Source: Sundog course extract, slide pages 224 and 225.

Action groups: function details or an OpenAPI schema

An action group defines what the agent can do for the user. You define the parameters the agent must collect for each action. You also choose how the agent sends those values to the executor. The two common definitions are function details, which name each function and its parameters, and an OpenAPI schema, which describes API operations with paths and methods.

Function details suit a few named actions that share one executor. An OpenAPI schema suits an API that already has a specification. The Sundog course says OpenAPI schemas can be uploaded to Amazon S3 or edited in the console. The schema tells the agent the operation names, parameter types, and required fields.

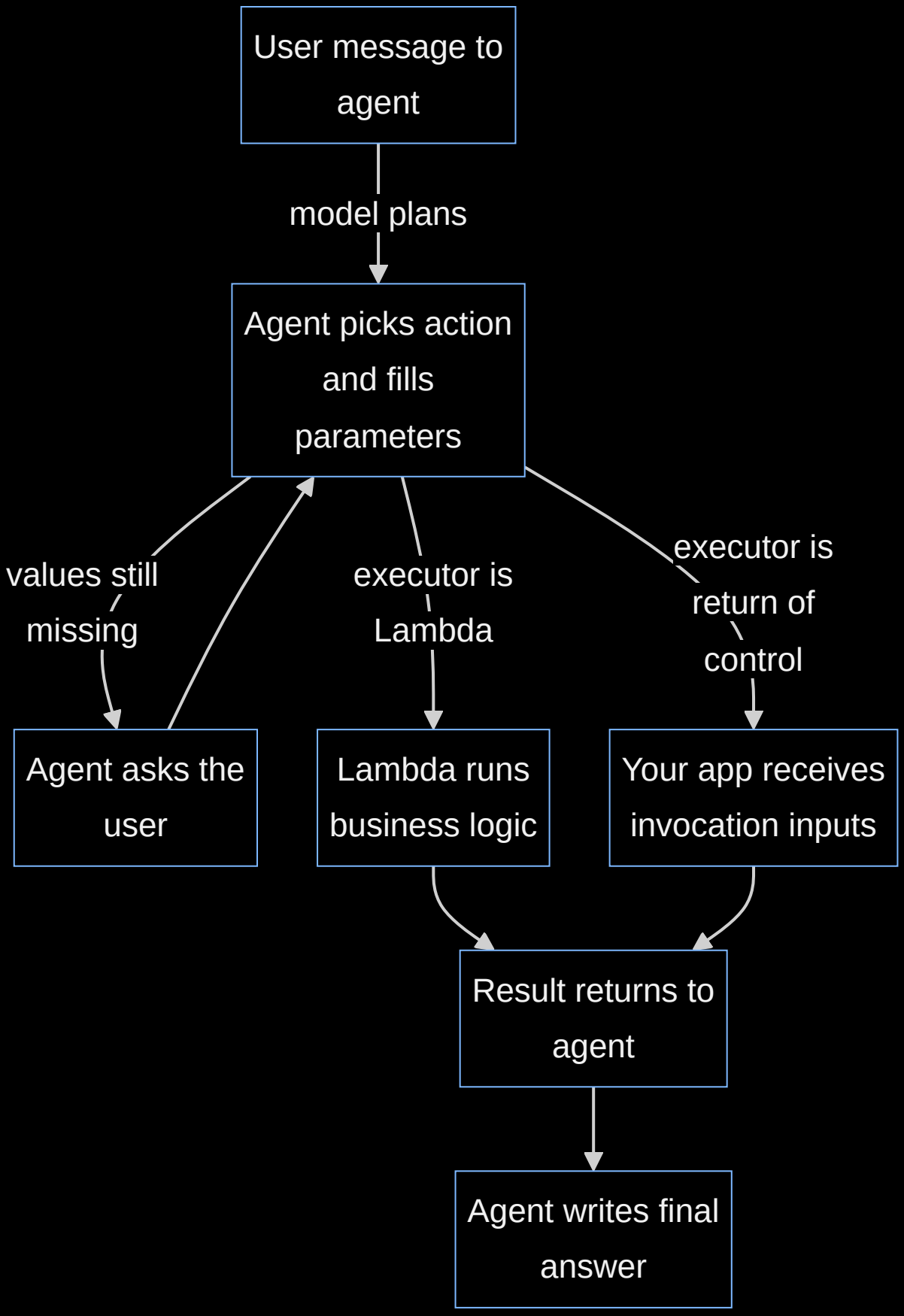


Figure: The model plans and fills parameters, then the executor runs the action and the agent writes the reply.

Rule: Use function details for a few named actions behind one executor. Use an OpenAPI schema when the API already has a spec.

Trap: Assuming the model runs the action. The agent only chooses the action and the values, and the executor does the work.

User confirmation is a per-action setting in an action group. It makes the agent ask the user before the action runs. I did not confirm the exact behavior of this setting in the pages fetched for this module, so verify it in the action group docs before you rely on it.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-action-create.html (action group purpose, elicited parameters). Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-returncontrol.html (function details and OpenAPI examples). Source: Sundog course extract, slide pages 224 and 258.

Return of control: your app runs the tool

Return of control changes the executor from Lambda to your application. You set the custom control value RETURN_CONTROL in the action group executor. When the agent chooses that action, the InvokeAgent response carries the invocation inputs and an invocation ID instead of calling a function.

Your application then runs the function, and it sends the result in the session state of another InvokeAgent request. The request must use the same invocation ID and action group that the response returned. The agent uses that result as context, and then writes the answer or chooses another action.

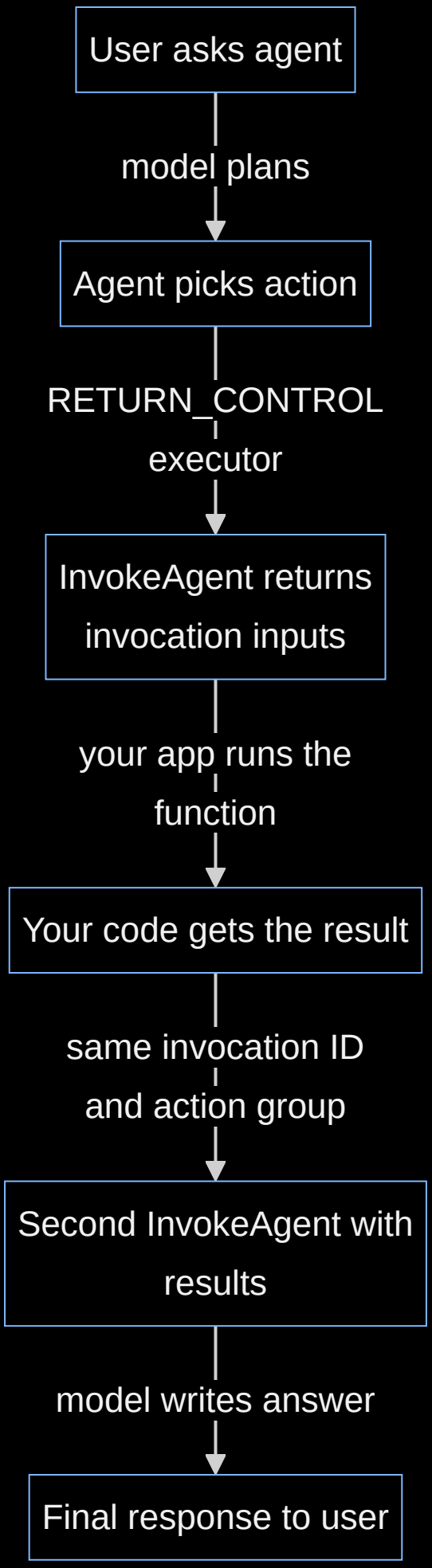


Figure: Return of control splits one tool call into two InvokeAgent requests with your code in the middle.

The return of control pattern also applies to collaborator agents. When a collaborator returns control, the supervisor response names the collaborator, and the caller sends results back with the matching agent identifier. The Sundog course presents this as the way to keep tools inside your own environment while the model still plans.

Rule: Choose return of control when tool execution must stay in your code or network, and the model must still plan the steps.

Trap: Choosing return of control to make tools faster. It adds a round trip to your application, so it usually adds latency and code.

Trap: Sending results with a new invocation ID. The agent matches the result to the pending call only with the ID and action group it returned.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-returncontrol.html (RETURN_CONTROL, invocationInputs, returnControlInvocationResults, collaborator examples).

Knowledge bases and agentic RAG

An agent can have knowledge bases attached. A prompt tells the model when to search each one, such as for questions about a named topic. The Sundog course calls this agentic RAG and says that retrieval is just another tool the agent can choose.

The design choice is whether the model should decide to retrieve. A fixed pipeline that always retrieves first is simpler and easier to test. An agent that retrieves only when needed can answer a mixed set of questions with fewer searches. Choose by how predictable the question mix is.

Rule: Attach a knowledge base to an agent when the model must decide when to search. Use a fixed retrieval step when every question needs it.

Trap: Treating a knowledge base as a per-user memory. It holds the documents you load, so it does not record the preferences of one customer across visits.

Source: Sundog course extract, slide pages 219 to 222 (knowledge bases and agentic RAG).
Source: Sundog course extract, slide page 234 (knowledge bases framed as a memory type).

Session state, memory, and what survives a session

Short-term memory holds the chat history and events of one session. Long-term memory keeps distilled knowledge from earlier conversations, such as a summary of a past visit or a stated preference. The Sundog course lists both types and says long-term data must be stored somewhere, such as a database.

Session state fits the active thread. A troubleshooting thread, a booking in progress, or a set of pending values belongs there. Long-term memory fits facts that must return next month. The two must be kept apart, or old data floods the active context and the model starts to mix up users.

Bedrock Agents has its own memory and session attribute features. I did not confirm their exact field names or retention rules in the pages fetched for this module, so those details are Not in source here. The design rule below holds for any memory store.

Rule: Keep active turn context in session state. Keep facts that must survive later sessions in a memory store scoped to the user.

Trap: Putting the full transcript in every prompt to get memory. Cost and context length grow each turn, and the design has no retention rule.

Source: Sundog course extract, slide page 234 (short-term and long-term memory) and slide page 243 (AgentCore short-term Session and Events, long-term Memory Records and strategies). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Memory component).

Versions, aliases, and deployment

An alias for an agent is a deployed snapshot that clients call. The Sundog course says an alias creates the deployed snapshot, and the application calls InvokeAgent with the alias ID and an agents runtime endpoint. The course also says on-demand throughput uses account quotas, while provisioned throughput buys a fixed rate for the agent. Exact quota numbers are Not in source.

Flows follow the same pattern. A working draft changes with each edit, a version is an immutable snapshot, and an alias points to the version that

applications call. The Flows overview says you can change the alias routing to revert to an older version or move to a newer one.

Rule: Point clients to an alias. Move the alias to roll back, and never point production clients to the draft.

Trap: Treating the draft as production. Each edit changes the draft, so a client on the draft sees untested behavior.

Source: Sundog course extract, slide page 225 (alias, on-demand and provisioned throughput, InvokeAgent). Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html (working draft, versions, alias, InvokeFlow).

Trace: the path of one agent session

Each agent response can include a trace. The trace shows the reasoning path for that invocation, including which knowledge base was hit, which action group was called, and any errors. The Sundog course lists trace types for preprocessing, orchestration, postprocessing, custom orchestration, routing classification, failure, and guardrail events.

You turn trace on by setting the trace flag in the InvokeAgent request. The return of control examples in the AWS docs set this flag in the same request. Store the trace with the final answer if reviewers must later see how the answer was made.

Exam tip: Words such as "which knowledge base was used" or "why did it choose that tool" point to agent trace. Words such as "who called the API" point to an audit log.

Trap: Choosing an API audit log to debug a wrong tool choice. An audit log shows that a call happened, not the path the agent took through the task.

Source: Sundog course extract, slide page 419 (Bedrock Agents trace types and trace content). Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-returncontrol.html (enableTrace in InvokeAgent requests).

Multi-agent collaboration: a supervisor and collaborators

In Bedrock Agents, you designate one agent as the supervisor and attach collaborator agents to it. The supervisor reads the user request, plans across the collaborators, and routes each task to the right specialist. Each collaborator can have its own instructions, tools, knowledge bases, and guardrails.

You must save the supervisor before you enable collaboration. You can add collaborators later. The supervisor uses the natural language description of each collaborator to route work, so the roles must not overlap. The number of collaborators allowed is Not in source for this module.

The Sundog course frames a supervisor, worker, and synthesizer pattern. It also warns that complex logic in one prompt becomes hard to maintain, and that simple sequences should stay as deterministic workflows. A supervisor helps when specialists need different tools and a shared conversation.

Rule: Use supervisor collaboration when several specialists must serve one conversation and one front door.

Trap: Giving one agent every tool because the team wants fewer agents. The Sundog course says a single agent with too many tools chooses poorly, so specialists with fewer tools fit better.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html (supervisor and collaborator roles, saving before collaboration, routing). Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-returncontrol.html (collaborator return of control). Source: Sundog course extract, slide pages 228 to 229.

AgentCore: the platform for any framework

Amazon Bedrock AgentCore is a platform for building, deploying, and operating agents with any framework and any foundation model. Its services can be used together or one at a time. AWS lists Runtime, Memory, Gateway, Identity, Code Interpreter, Browser, Observability, Policy, Evaluations, and other services. Pricing is consumption based with no minimum fee in the overview.

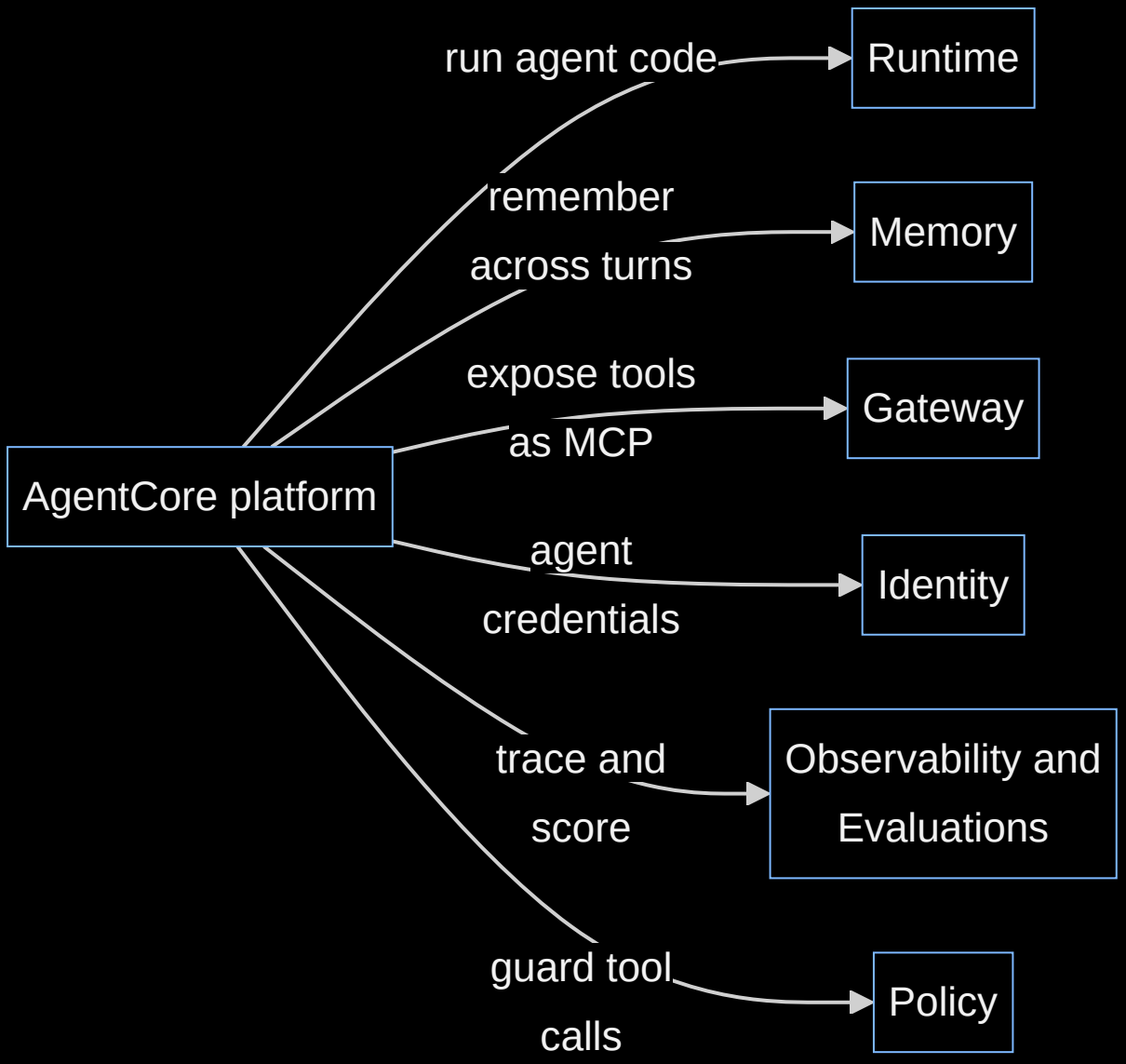


Figure: Each AgentCore service answers one operational question, and you can use any subset.

Rule: Choose AgentCore when the framework or model is not fixed to Bedrock Agents, or when the agent needs a runtime that suits extended asynchronous work.

Trap: Reading "AgentCore" as a single product that must be used whole. AWS says each service works alone, so the question may need only one of them.

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (core services table, framework statement, pricing). Source: Sundog course extract, slide pages 240 and 241.

AgentCore Runtime: hosting agent code

Runtime is a serverless environment for deploying and scaling agent code. AWS says it has fast cold starts, extended runtime support for asynchronous agents,

session isolation, and built-in identity. It works with Strands Agents, LangGraph, and other frameworks, and with any model, and it supports MCP and A2A protocols.

The Sundog course says you can deploy with the AgentCore command line, which uses CodeBuild, or build your own container. A deployed runtime can have more than one endpoint, and an observability dashboard shows usage and performance. Container build details beyond these points are Not in source.

Rule: Choose Runtime when you want to host agent code without managing containers, clusters, or load balancers yourself.

Trap: Choosing Step Functions to host the agent code. Step Functions coordinates calls and states, but it does not host a framework agent loop.

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Runtime description). Source: Sundog course extract, slide page 242 (serverless endpoints, CLI, multiple endpoints).

AgentCore Memory: short-term events and long-term records

AgentCore Memory has two parts. Short-term memory keeps the raw events of a session, including each message and each tool result. Long-term memory stores records extracted from those events, such as preferences, facts, and session summaries, under configured strategies.

Scope memory to the person and to the conversation. A person-scoped record survives many sessions. A session-scoped event stays with one visit. The exact identifiers and the strategy names are Not in source for this module, so read the memory docs before you write code.

Rule: Use Memory when durable records per person must come back in later sessions with a managed store.

Trap: Relying on short-term events to recall a fact from last month. Events belong to the session, and the durable recall comes from long-term records.

Source: Sundog course extract, slide page 243 (short-term session and events, long-term memory records and strategies). Source: https://docs.aws.amazon.com/bedrock-

agentcore/latest/devguide/what-is-bedrock-agentcore.html (Memory description, works with Strands and LangGraph).

AgentCore Gateway: tools as MCP for many clients

Gateway turns APIs, Lambda functions, and existing services into Model Context Protocol tools. It also connects to existing MCP servers. The Sundog course lists targets as OpenAPI REST APIs, Smithy models, endpoints, and Lambda functions. It also says Gateway manages OAuth security and can do semantic tool selection.

Gateway solves the problem of many agents needing the same tools. You register the tool source once, and each client connects to the Gateway endpoint. This avoids a custom adapter in each agent. The Sundog course says Gateway is the layer for using external tools at scale.

Rule: Put shared tools behind Gateway when several agents or frameworks call the same tools.

Trap: Treating every Gateway target as an MCP server. The target type decides whether the Gateway aggregates tools as MCP or only proxies the call, and the pages I read do not give that detail.

Source: Sundog course extract, slide page 246 (Gateway targets, MCP conversion, OAuth, semantic tool selection). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Gateway description, connect existing MCP servers).

AgentCore Identity: the own identity of the agent

Identity manages the identity of the agent itself. It controls how the agent reaches outside tools and AWS resources, stores credentials, and supports OAuth 2.0. The Sundog course says it works with common identity providers and lists built-in support for several SaaS providers.

The course stresses that this is different from end-user sign-in. A customer who logs in to your web app is one identity. The agent that calls a CRM API is another identity. Identity answers the second question.

Rule: Use Identity when the agent must call outside services and the credentials must be stored and rotated in one managed place.

Trap: Confusing the agent identity with the end-user login. Identity does not replace the user sign-in in your web app.

Source: Sundog course extract, slide page 247 (agent identities, OAuth 2.0, credential storage, difference from user OAuth). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Identity description and identity providers).

AgentCore Observability and Evaluations

Observability gives a unified view of the steps in an agent run. It uses OpenTelemetry-compatible telemetry and shows the execution path, intermediate outputs, and failures. Evaluations scores agents from sessions, traces, and spans, and it can use traces from Strands Agents or LangGraph instrumented with OpenTelemetry or OpenInference.

The Sundog course says evaluations check whether the agent performs tasks, handles edge cases, and keeps consistent output. It also says results show in CloudWatch. The modes for live sampling, targeted spans, and offline sets are Not in source for this module, so check the evaluation types page before you choose a mode.

Rule: Use Observability to inspect one run. Use Evaluations to score many runs against stated criteria.

Trap: Using CloudWatch Logs alone to decide if a task succeeded. Logs show events, but they do not score goal completion or tool choice.

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Observability and Evaluations descriptions). Source: Sundog course extract, slide page 250 (AgentCore Evaluations).

AgentCore Browser and Code Interpreter

Browser is a managed cloud browser runtime. An agent can navigate sites, fill forms, and extract information. It works with Playwright and similar automation

frameworks. Code Interpreter is an isolated sandbox where an agent runs Python, JavaScript, or TypeScript to compute results.

Use Browser when the only way to reach the data is a website or a web form. Use Code Interpreter when the answer needs a calculation or a chart that the model writes as code. Both run outside your application, which keeps generated code and browsing away from your own servers.

Rule: Choose Browser for a site with no usable API. Choose Code Interpreter for generated calculations or charts.

Trap: Using Code Interpreter for a fixed formula. A Lambda function with the known formula is simpler and more predictable.

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Browser and Code Interpreter rows). Source: Sundog course extract, slide page 244 and slide page 223 (optional code interpreter in Bedrock Agents).

AgentCore Policy: a gate on each tool call

Policy sets deterministic boundaries for agents. It integrates with Gateway and intercepts each tool call before it runs, then checks the call against rules. Rules are written in Cedar or a compatible policy language, and they can say which tools an agent can call, which actions it can take, and under what conditions.

The Sundog course says Policy is evaluated outside the agent code. That matters when the model must not be able to override a rule through its prompt. Fine-grained control answers who can do what under what conditions.

Rule: Use Policy when a rule must hold even if the model asks for a forbidden tool call.

Trap: Writing the rule only in the system prompt. A prompt is advice to the model, not an enforced gate.

Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Policy row: intercepts tool calls, Cedar-compatible). Source: Sundog course extract, slide page 248 (Policy in AgentCore, evaluated outside agent code).

Bedrock Flows: a visual chain of prompts and models

Bedrock Flows links prompts, foundation models, knowledge bases, Lambda functions, and other AWS services into a workflow. You build it in the visual builder or through the APIs. A flow is made of nodes and connections, and connections can be conditional. The Sundog course notes that Prompt Flows was absorbed into the larger Flows feature.

Flows have a working draft, immutable versions, and aliases. Applications call InvokeFlow on an alias. The loop and iterator node details, and any loop limits, are Not in source for this module.

Flows fit a chain that product owners or analysts must change without application code. The steps are mostly fixed, and each step is a prompt, a model call, a condition, or a service call. A flow is a good place for prompt chaining and routing by a condition. It is not a managed open-ended agent.

Rule: Choose Flows when the steps are mostly fixed and non-developers must edit them visually.

Trap: Choosing Flows for an open-ended task that needs the model to decide new steps. The flow only follows the connections you drew.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html (overview, visual builder, versions, aliases, InvokeFlow, async executions). Source: Sundog course extract, slide pages 110 to 113.

Step Functions: state machines with waits and audit

Step Functions runs state machines. A Task state calls an AWS service. Standard workflows run up to one year, follow an exactly-once model, and keep a full execution history for up to 90 days after the run. Express workflows run up to five minutes and use at-least-once or at-most-once execution, depending on how they are started.

Use Step Functions when you need auditable state transitions, retry and failure isolation, parallel branches, and explicit approval steps. The Sundog course lists these reasons and notes that serverless choices other than Step Functions also

exist. The model does not pick the steps here, because your state machine does.

Rule: Use Step Functions when your code must own the steps, the retries, and the wait for a person.

Trap: Using Step Functions to decide open-ended next steps for a model. The state machine follows the states you wrote, so it does not reason about new paths.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html (duration, execution semantics, history). Source: Sundog course extract, slide page 661 (auditable transitions, retry, approval).

Step Functions: callbacks, task tokens, and heartbeats

The wait for a callback uses the resource suffix for waiting on a task token. The task token is read from the Context object, and the workflow pauses until the same token returns with a success, a failure, or a heartbeat call. The callback APIs are SendTaskSuccess, SendTaskFailure, and SendTaskHeartbeat.

Set HeartbeatSeconds on a waiting task to detect a missing reply. If no valid token returns in that interval, the task fails with the States.Timeout error name. A timed-out task gets a new random token. A callback token must return from a caller inside the account of the workflow, or the reply is refused.

A Lambda task, Amazon SQS, Amazon SNS, and EventBridge can all carry the token to an outside system in the optimized integrations. Standard workflows support the callback pattern, and Express workflows do not. The wait itself is the callback pattern, so no polling loop is needed.

Rule: Use a callback task when a person or outside system must reply later and no service job is running.

Trap: Using the run-a-job pattern for human approval. The run-a-job pattern waits for a service job to finish, and a reviewer is not a service job.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html (three patterns, callback tokens, HeartbeatSeconds, same-account rule, optimized integration

table). Source: https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html (callback support by workflow type).

Step Functions: Map state and large fan-out

A Map state runs the same steps for each item in a list. The Distributed Map mode runs each item as its own child workflow execution, and it is supported in Standard workflows only. The Standard and Express comparison page marks Distributed Map as not supported in Express.

Use Distributed Map when the item count is large, such as tens of thousands of files, and each item must succeed or retry on its own. The Map state lets you set how many children run at once. The exact concurrency limit and item count limit are Not in source for this module.

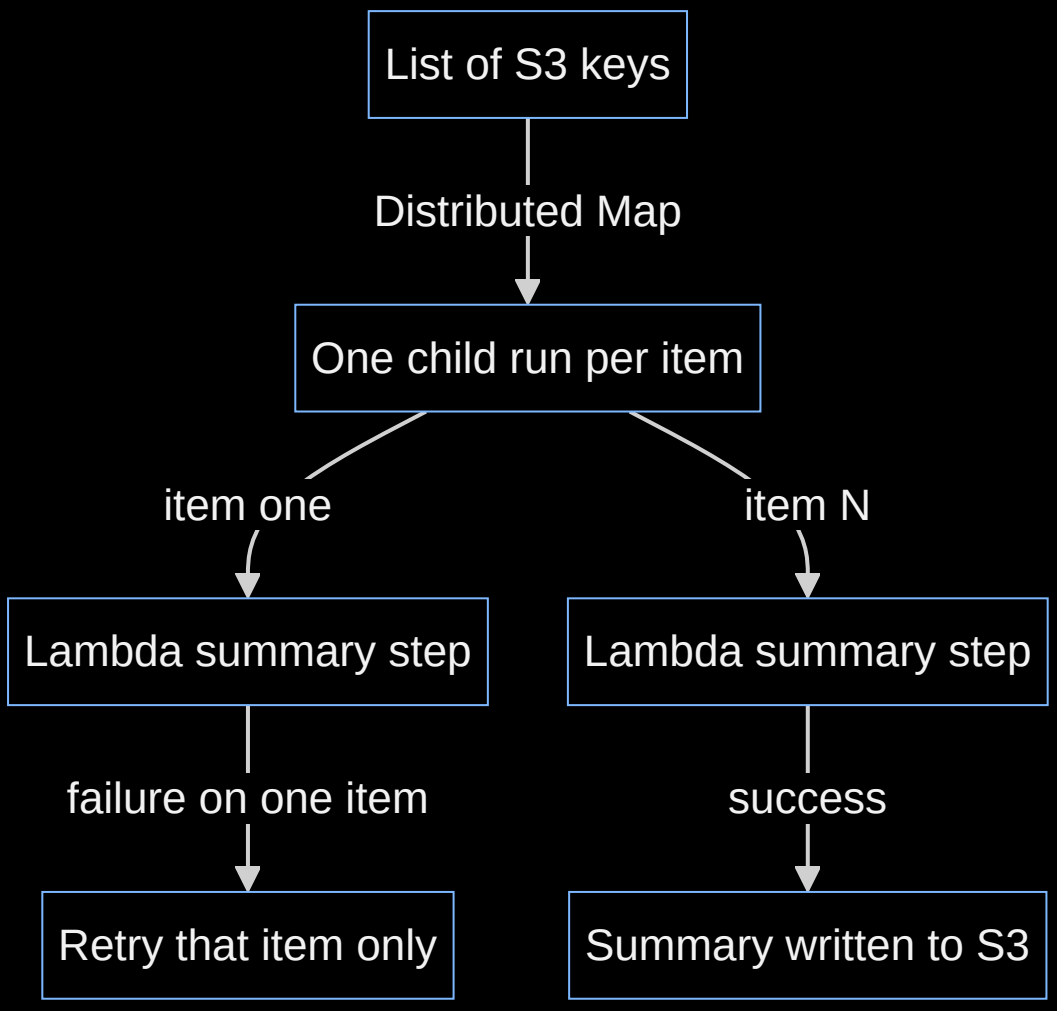


Figure: Each item runs in its own child, so a failure retries one item and not the whole batch.

Rule: Use Distributed Map in Standard for large item sets that need per-item retries and history.

Trap: Using one Lambda function to loop over every item. A loop inside one function hits the Lambda timeout and loses per-item status.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html (Distributed Map supported in Standard, not Express). Source: Sundog course extract, slide page 295 (parallel branches in Step Functions).

Step Functions: retries, catches, and fallbacks

A Retry field on a task retries an error with backoff. A Catch field routes an error that remains after the retries to another state, such as a fallback path. The fields for jitter and maximum delay also exist in the state language, but I did not confirm their exact names and defaults in the pages fetched here, so verify them in the state definition docs.

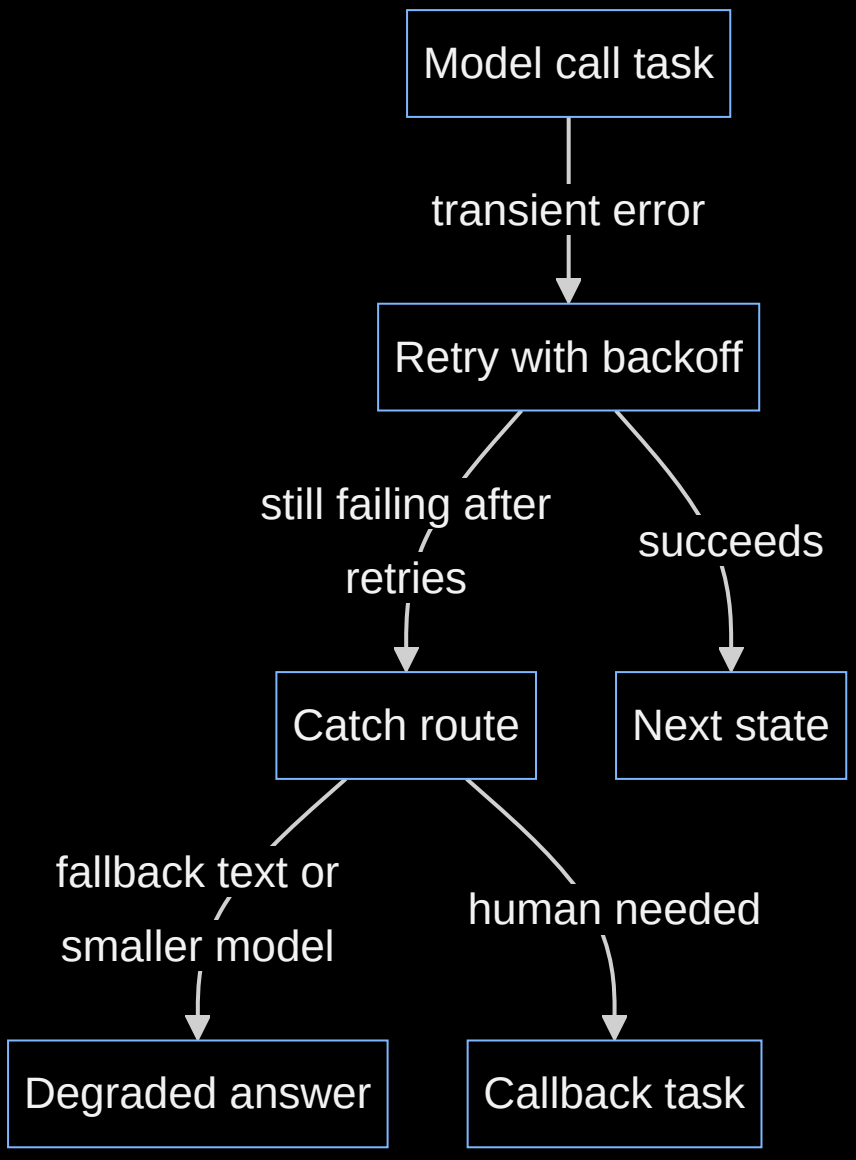


Figure: Retry handles short failures, and Catch decides what the workflow does when retries run out.

Retry alone does not stop a shared outage from causing a retry spike. Add jitter so many executions do not retry at the same moment. Add a bounded attempt count, and a fallback path that returns a safe answer.

Rule: Use Retry for short transient errors and Catch for the fallback path after retries run out.

Trap: Unlimited retries with no jitter during a shared outage. Every execution retries at the same moments, and the outage gets worse.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html (Bedrock optimized integration, task definition with a Bedrock Task). Retry and Catch field details: Not in source in the pages fetched for this module.

Step Functions with Amazon Bedrock

Step Functions has an optimized integration that invokes a Bedrock model. The InvokeModel request body can be up to 256 KiB. For larger payloads, AWS recommends the Input field, which reads the request from an S3 URI, and the Output field, which writes the response to S3. You set either Body or Input, but not both.

Bedrock actions support Request Response for model calls. The customization job API also supports the run-a-job pattern. The callback pattern is available for Bedrock in Standard workflows. The integration with AgentCore supports Request Response only.

Keep large prompts and model outputs in S3, and pass references through state data. This keeps state small and avoids the payload limit. A workflow that carries full transcripts in state will hit size limits and cost more to store.

Rule: Use S3 Input and Output for large prompts or outputs, and pass only references in state data.

Trap: Putting a long transcript in the state data of every step. The payload limit and the storage cost both grow with the transcript.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html (256 KiB Body limit, Input and Output S3 fields). Source: https://docs.aws.amazon.com/step-

functions/latest/dg/connect-to-resource.html (optimized integration table: Bedrock, AgentCore).

AWS Lambda: the fifteen minute limit

Lambda timeout defaults to 3 seconds and can be set up to 900 seconds, which is fifteen minutes. For Lambda Managed Instances, asynchronous and event source invocations allow up to 5,400 seconds. The fifteen minute limit applies to the standard function timeout, so a longer job must be split.

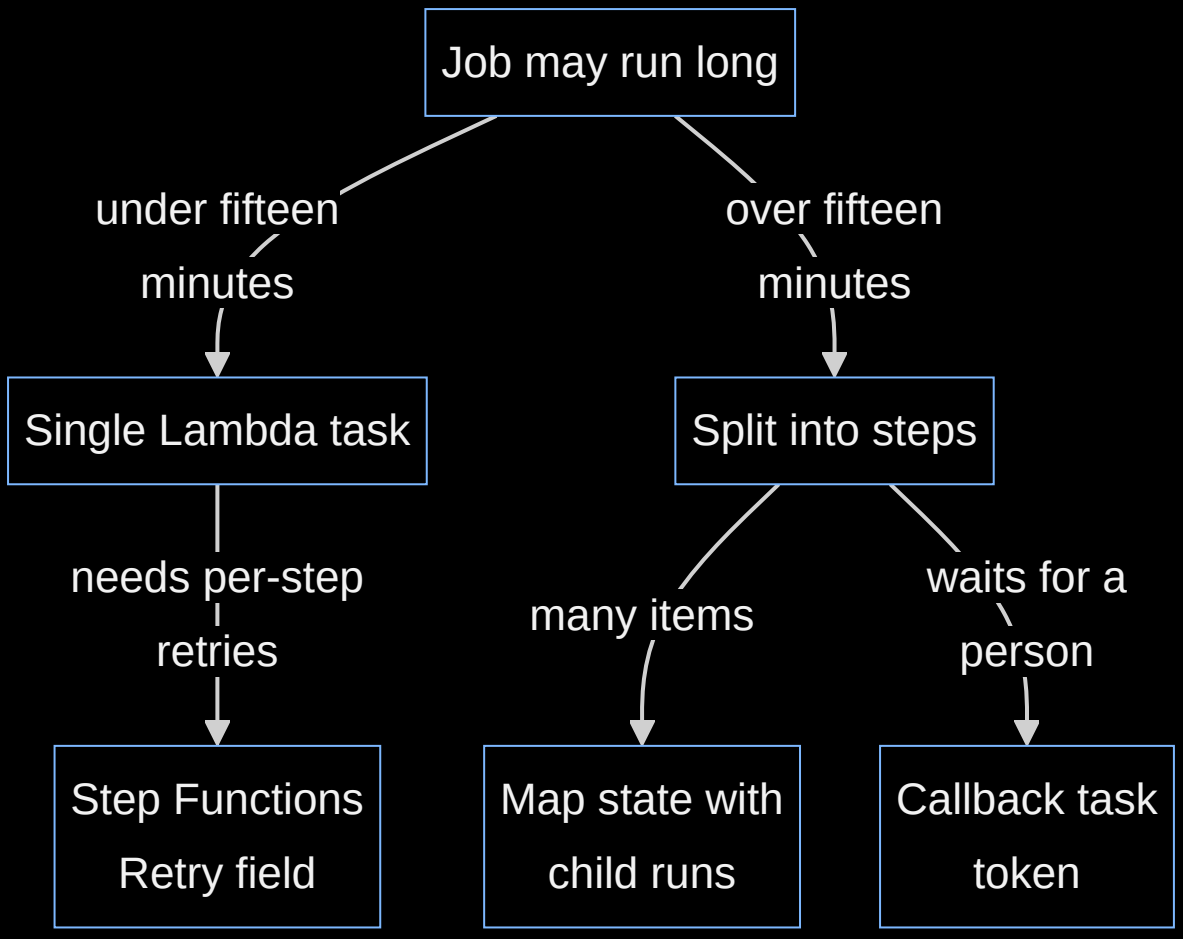


Figure: A job over the limit is split into steps, and Step Functions carries the state between them.

Lambda suits short, stateless work such as one tool call, one transformation, or one API request. It is a poor place to hold a long model loop or a long wait. Raising the timeout or memory does not fix a loop that never ends.

Rule: If the job can run longer than fifteen minutes, split it into steps and move the wait to Step Functions.

Trap: Raising the Lambda timeout to fix an agent that loops forever. The loop keeps calling the model and the tools, so cost grows and the answer does not

improve.

Source: https://docs.aws.amazon.com/lambda/latest/dg/configuration-timeout.html (default 3 seconds, maximum 900 seconds, Managed Instances limit).

EventBridge and SQS: decoupling producers from consumers

Amazon EventBridge routes events from producers to targets through rules, and it can start a workflow on a schedule. Amazon SQS holds messages in a queue until a consumer reads them. A queue smooths spikes, and a dead-letter queue holds messages that keep failing.

Use a queue when producers can outpace consumers, such as uploads that arrive all day but processing that runs at night. Use EventBridge when one event must reach several targets. Use both when a schedule reads a queue and starts a batch workflow.

Trap: Using EventBridge as a durable work queue for every item. EventBridge routes events, and a queue is the right buffer when items must wait for a consumer.

Source: No AWS page for EventBridge or SQS was fetched for this module, so the numeric limits in this subsection are Not in source. Source: Sundog course extract, slide page 661 (approval and retry as reasons to use Step Functions).

MCP servers and Streamable HTTP

The Model Context Protocol is a standard interface for agent to tool calls. Its data layer uses JSON-RPC 2.0. A server can expose tools, resources, and prompts. The Sundog course says the transports are stdio and HTTP streaming, and the MCP specification names the HTTP transport Streamable HTTP. The spec version details are Not in source.

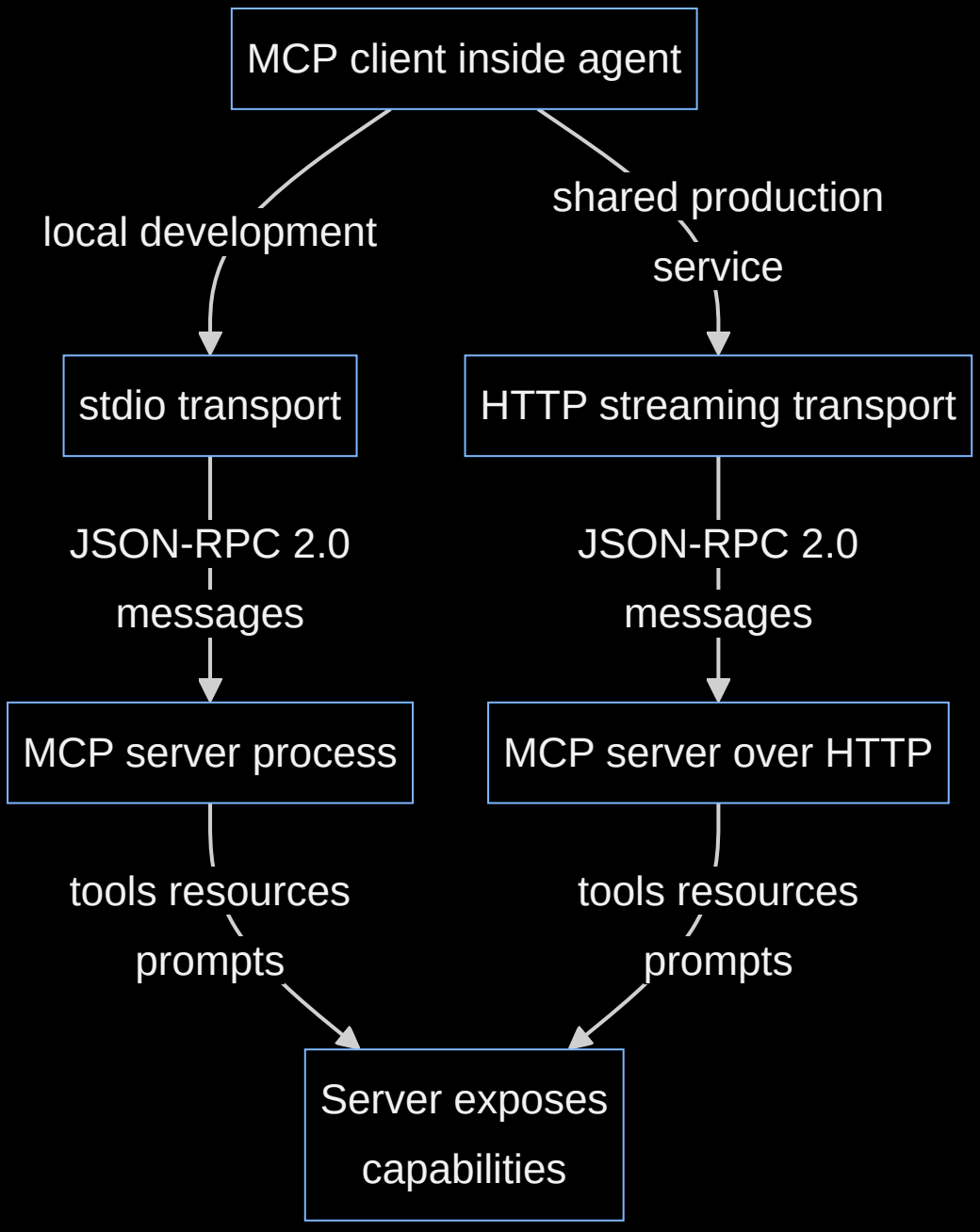


Figure: One protocol, two transports, and the same tools and resources at the end.

The Sundog course gives deployment advice. A stateless, lightweight MCP server fits AWS Lambda. A complex server with long-running work or heavy memory fits Amazon ECS or AWS Fargate. API Gateway can present MCP endpoints too, and AgentCore can host MCP servers. The course also notes that AWS guidance for MCP architecture differs from some exam guide wording, so use the current AWS docs.

Rule: Use Lambda for stateless MCP tools. Use ECS or Fargate when the server must keep state or load a large engine.

Trap: Putting a server that loads a large engine and keeps session state into Lambda. Each cold start reloads the engine, and state does not persist between

calls by design.

Source: Sundog course extract, slide page 256 (MCP data layer, JSON-RPC 2.0, stdio and HTTP streaming, servers expose tools resources prompts) and slide page 257 (Lambda, ECS, Fargate, API Gateway, AgentCore for MCP). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Gateway MCP conversion, Runtime MCP support).

Strands Agents and AWS Agent Squad

Strands Agents is an open-source Python SDK from AWS for building agents and multi-agent systems. Its agent loop selects tools, runs them, and returns the response. It has built-in tools, supports MCP, and works with models outside AWS. The Sundog course says Strands can use Bedrock models, Lambda, and Step Functions.

AWS Agent Squad is an open-source framework for multi-agent workflows. It routes each query to the right agent with a classifier, shares conversation context across agents, and integrates with Bedrock Agents. The Sundog course describes it as focused on routing, while Strands focuses on tool use inside one agent loop.

Rule: Use Strands for the tool loop of one agent or for agents used as tools. Use Agent Squad to route a request across separate services.

Trap: Choosing Agent Squad to run the tool loop of one agent. Agent Squad routes between agents, and the tool loop belongs to the agent framework.

Source: Sundog course extract, slide pages 235 to 239 (Strands Agents agent loop, built-in tools, MCP support, Agent Squad routing and Supervisor Agent).

Deterministic workflow versus agentic reasoning

A deterministic workflow has steps you can list before the request. Its branches use known values, such as a score or a status field. An agentic design lets the model choose the next step from a result it has not seen yet. The Sundog course says complex decision making can benefit from agents, but simple cases should stay as deterministic workflows.

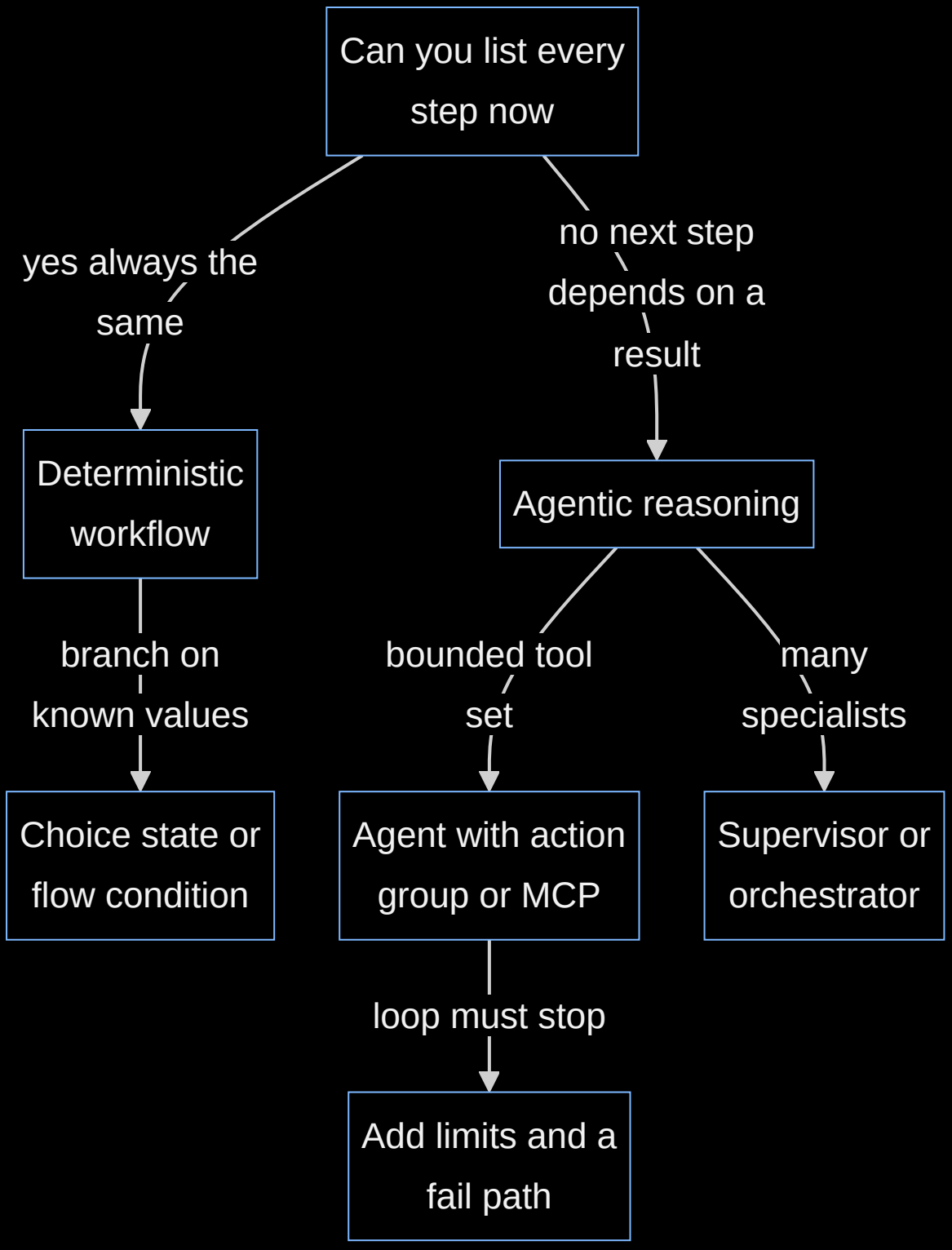


Figure: Pick the workflow first, and add an agent only where the next step truly depends on a result.

Rule: Use a workflow for known steps and an agent for steps the model must choose. Add a loop limit to every agent.

Trap: Using an agent for a three-step process with fixed rules. The model adds cost and variation that the rules do not need.

Source: Sundog course extract, slide page 227 and slide page 229 (use multi-agent only when complex decisions need it, otherwise keep deterministic workflows). Source:

https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html (flows as chained steps).

Distinguishing the confusable pairs

The pairs below cover the choices that cause most wrong answers in this area. For each pair, read the decision tree, answer the one question, and then check the trap. The comparison rows use three fixed labels: purpose, input, and output or effect.

Pair 1: Converse tool use versus a Bedrock Agents managed loop

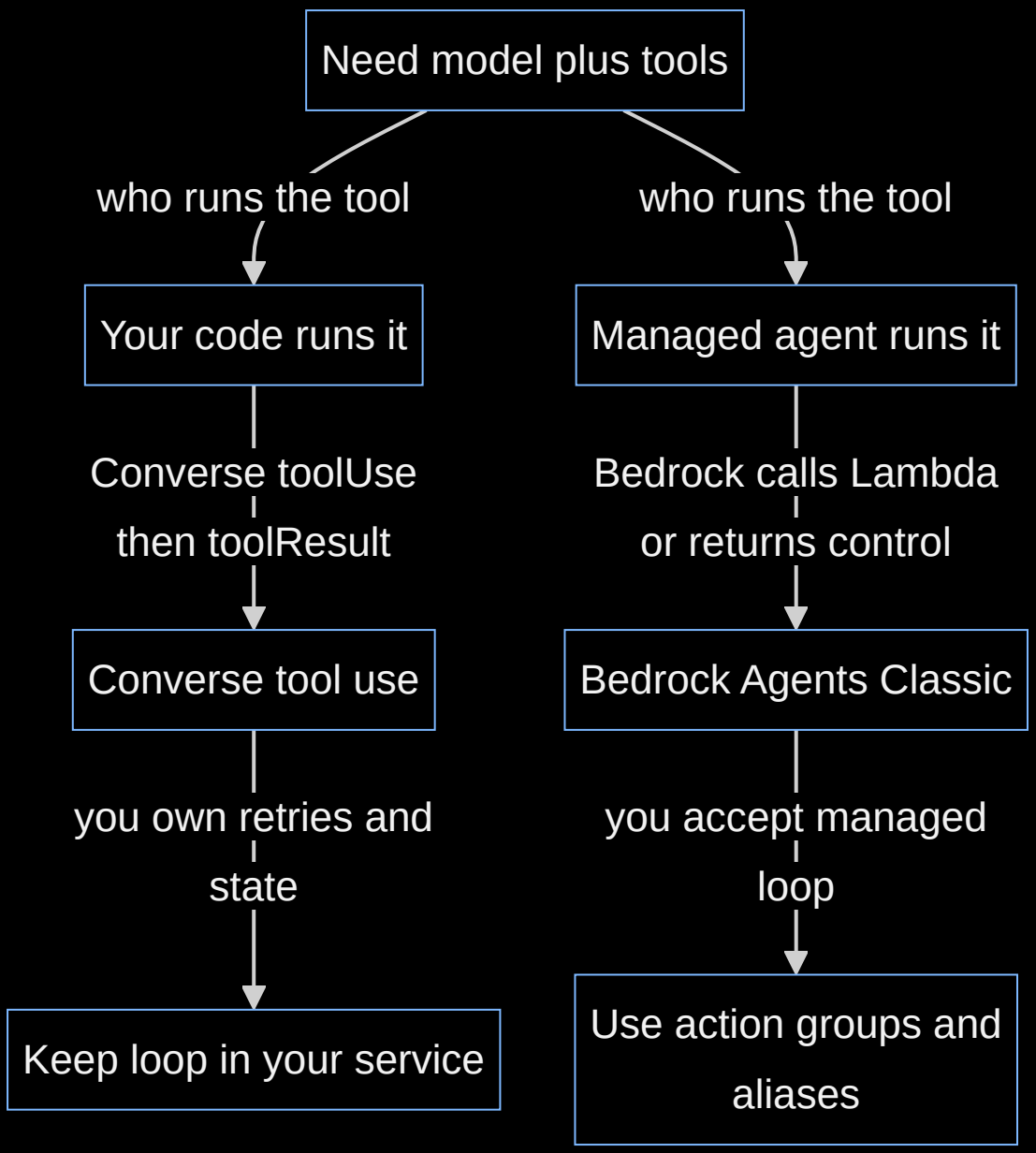


Figure: The owner of the tool call decides between the two patterns.

- **Purpose:** Converse tool use gives you a loop you control. Bedrock Agents gives you a loop that Bedrock runs.

- **Input:** Converse takes messages and tool definitions in your request. Bedrock Agents takes a user message, action groups, and optional knowledge bases.
- **Output or effect:** Converse returns tool requests that you must answer. Bedrock Agents returns a final answer, with a trace if you ask for it.

Exam tip: Look for "your application must run the tool" or "your code must retry". Those phrases point to Converse. Look for "managed" or "action group" to point to Bedrock Agents.

Rule: If your code must own each tool call, use Converse. If Bedrock should own the loop, use Bedrock Agents Classic or AgentCore.

Trap: Picking Bedrock Agents because the question says tool use. The option may be valid, but the question must also say the managed loop is acceptable.

Source: Sundog course extract, slide pages 20 to 21 (Converse and toolConfig) and slide page 224 (action groups for the agent). Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-returncontrol.html (the split between managed execution and return of control).

Pair 2: Bedrock Agents Classic versus AgentCore

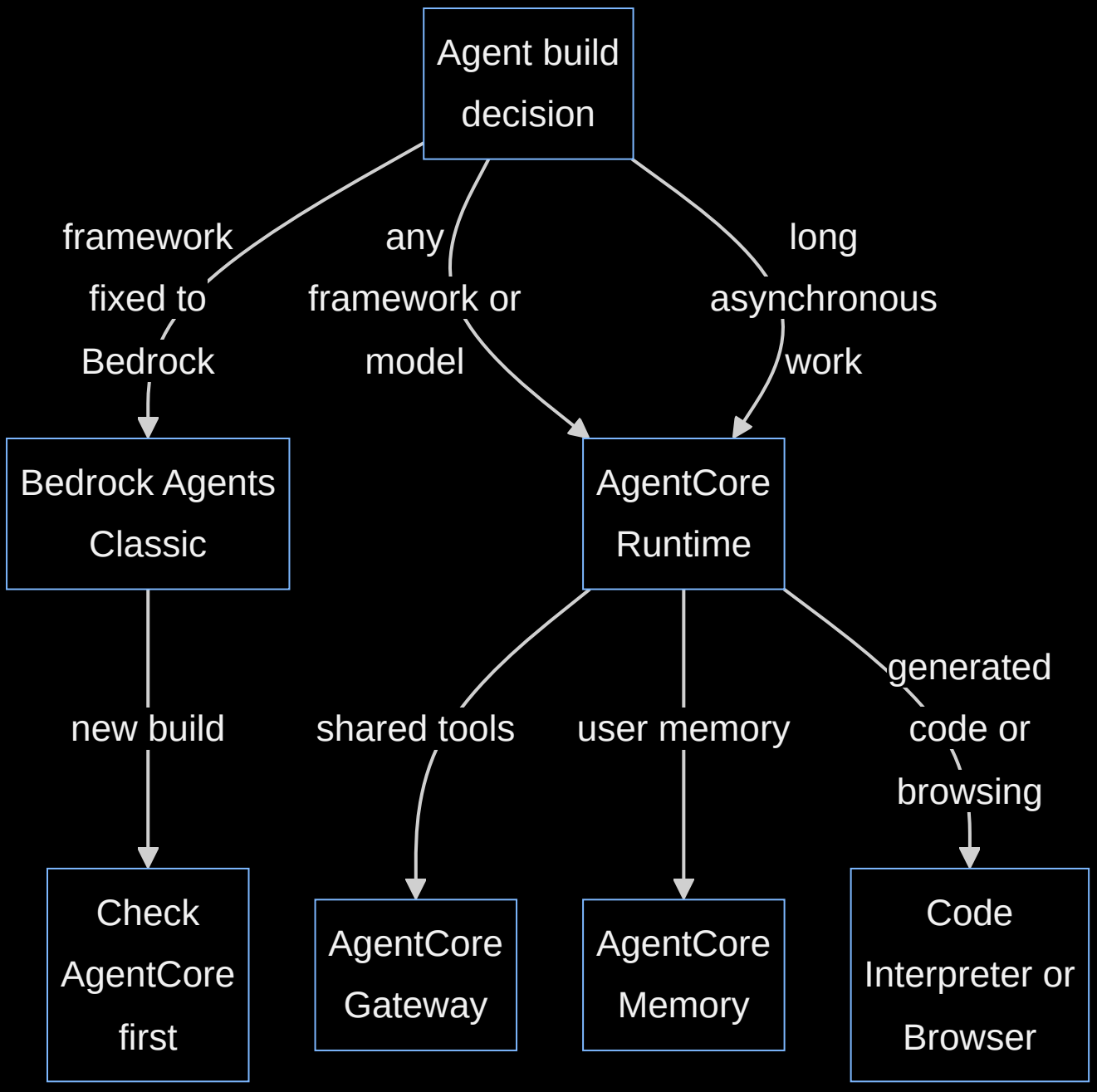


Figure: Choose the platform by framework freedom and by which operational service the agent needs.

- **Purpose:** Bedrock Agents Classic runs one managed agent inside Bedrock. AgentCore hosts and operates agents built with any framework.
- **Input:** Bedrock Agents takes action groups, knowledge bases, and aliases. AgentCore takes your agent code, plus any mix of Gateway, Memory, Identity, and Policy.
- **Output or effect:** Bedrock Agents returns answers from its managed loop. AgentCore returns answers from your code, with the platform services around it.

Exam tip: The words "any framework", "any model", or "long asynchronous work" point to AgentCore. AWS says Bedrock Agents Classic is no longer open to new customers.

Rule: For a new build with a chosen framework, start from AgentCore. Keep Classic only where it already runs.

Trap: Choosing AgentCore for a plain managed agent with two action groups when the question asks for the simplest managed option. AgentCore adds design work that the question did not ask for.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-action-create.html (maintenance note). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (any framework, any model, Runtime). Source: Sundog course extract, slide pages 240 and 245.

Pair 3: Bedrock Flows versus Step Functions

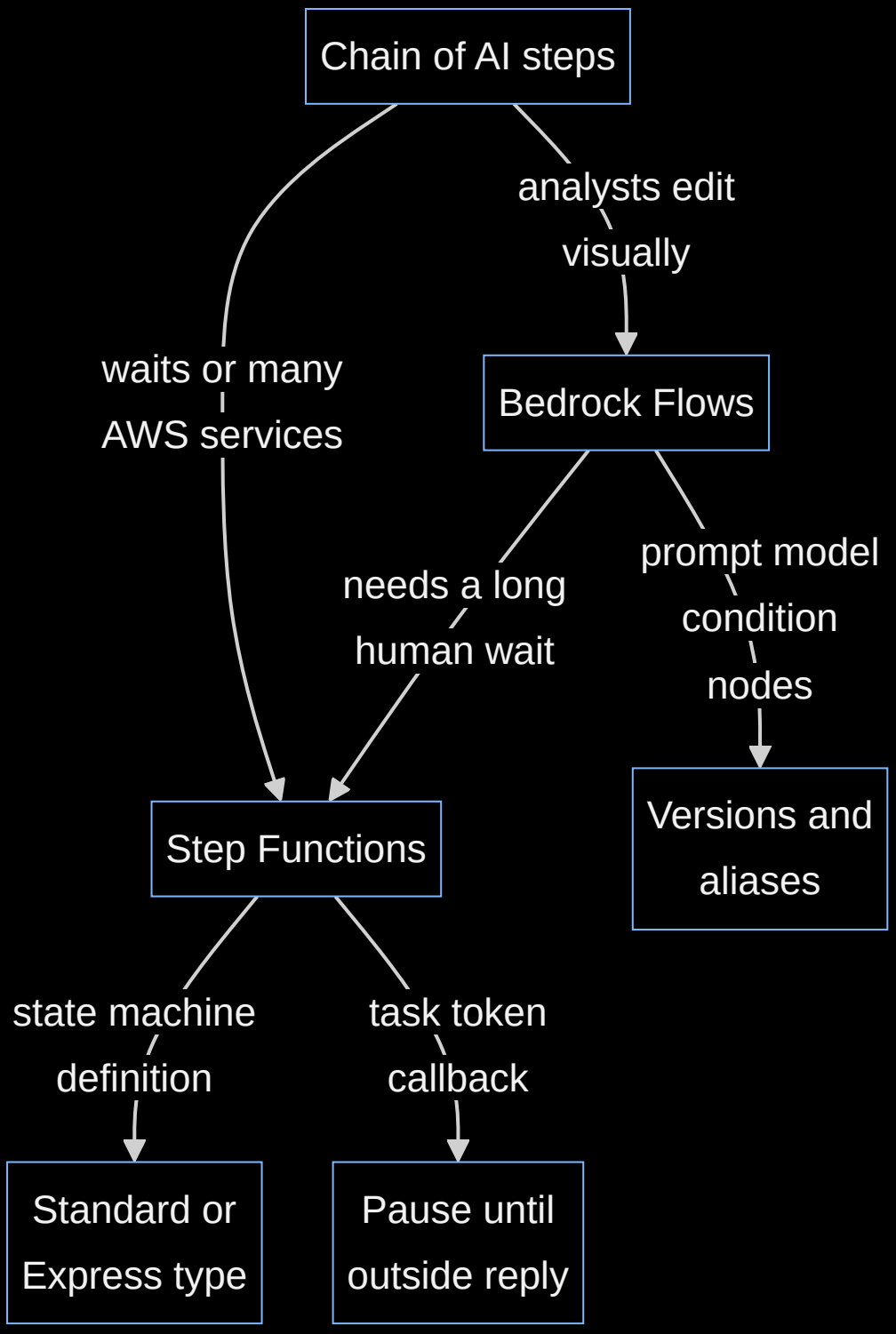


Figure: Flows is the visual prompt chain, and Step Functions is the durable state machine for waits and broad service calls.

- **Purpose:** Flows chains prompts, models, and conditions for AI steps. Step Functions coordinates any AWS services with durable state.
- **Input:** Flows takes a flow input and node outputs. Step Functions takes a JSON input and the state machine definition.
- **Output or effect:** Flows returns the output of the last node. Step Functions records each state transition and can pause for a callback.

Exam tip: Words such as "visual builder for prompt steps" point to Flows. Words such as "wait for approval", "audit each state", or "call many AWS services" point to Step Functions.

Rule: Choose Flows for a visual prompt chain. Choose Step Functions when the chain must wait or coordinate many services.

Trap: Choosing Flows for an approval that takes days. The wait requirement in the question is the signal for a durable state machine.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html (visual builder, versions, aliases). Source: https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html (callback pattern). Source: Sundog course extract, slide pages 110 to 113 and slide page 661.

Pair 4: Standard workflow versus Express workflow

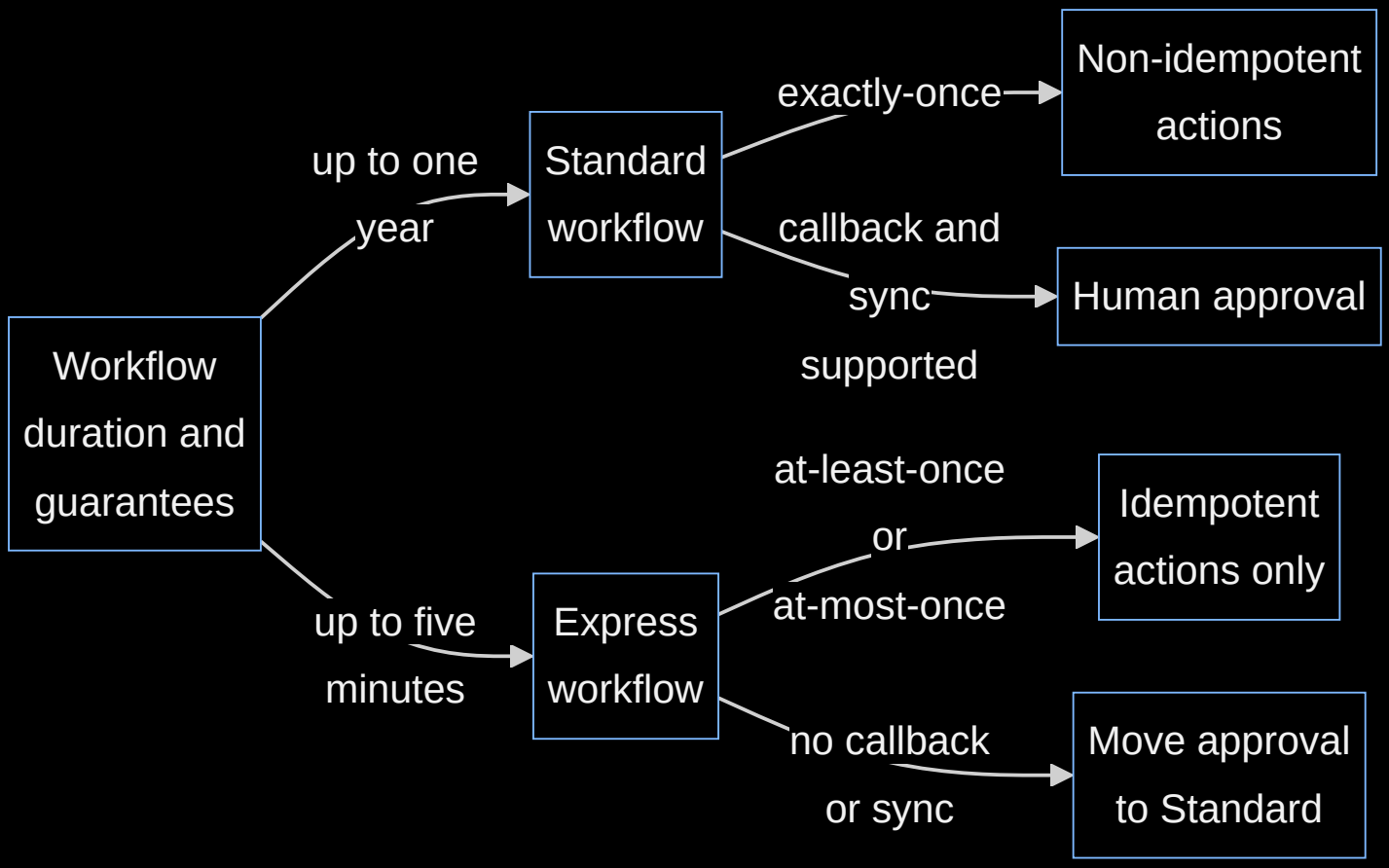


Figure: Duration and the callback rule decide the workflow type before cost does.

- **Purpose:** Standard runs long, durable, auditable workflows. Express runs short, high volume workflows.

- **Input:** Both take a state machine definition and an execution input. Express also allows synchronous start from API Gateway or Lambda.
- **Output or effect:** Standard keeps history for up to 90 days and bills per state transition. Express bills per execution, duration, and memory, and it does not keep history in Step Functions.

Exam tip: A wait longer than five minutes, or any callback, means Standard. Short bursts of idempotent work mean Express.

Rule: Use Standard for any step that waits for a person, a callback, or more than five minutes.

Trap: Keeping the approval in an Express workflow and looping over repeated runs to poll. The polling adds state, retries, and cost that Standard avoids.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html (duration, execution semantics, callback support, history, billing). Source: https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html (integration pattern support by workflow type).

Pair 5: Request Response versus run-a-job versus callback task token

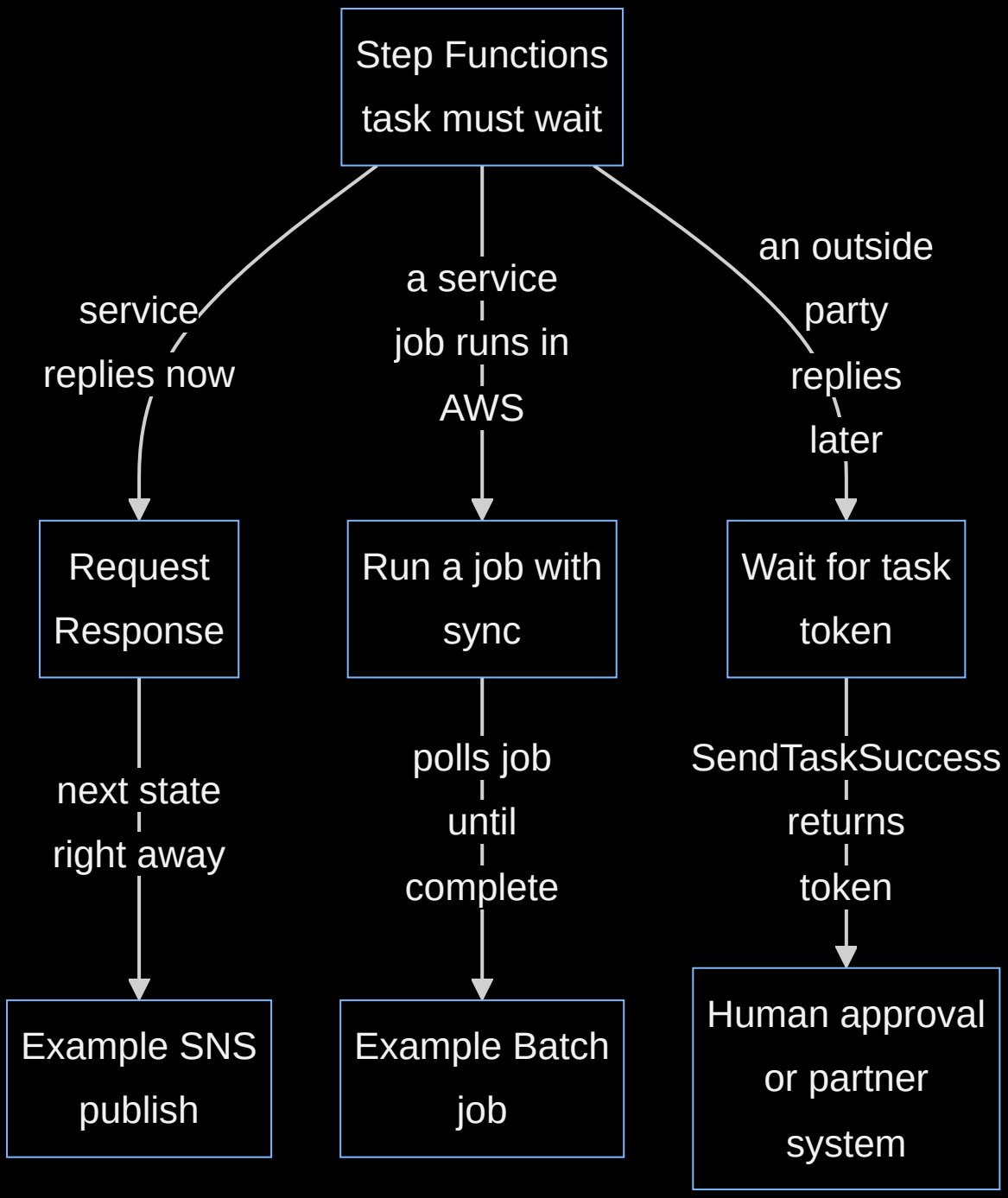


Figure: The reply source decides the pattern, and the service API does not.

- **Purpose:** Request Response moves on after the call returns. Run-a-job waits for a job to finish. The callback pattern waits for an external reply that carries the token.
- **Input:** Request Response takes the service parameters. Run-a-job takes the job parameters. The callback takes the parameters plus a token from the Context object.
- **Output or effect:** Request Response continues at once. Run-a-job continues when the job completes. The callback continues on SendTaskSuccess,

SendTaskFailure, or a heartbeat timeout.

Exam tip: The phrase "wait for a person" or "reply from another system" points to the callback. The phrase "wait until the batch job ends" points to run-a-job.

Rule: Use the callback pattern when the workflow must pause for an outside reply, and set a heartbeat.

Trap: Using run-a-job for a reviewer. Run-a-job expects a service job whose status the service can read, and a person is not that kind of job.

Source: https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html (three patterns, callback, run-a-job, HeartbeatSeconds). Source: https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html (pattern support).

Pair 6: AgentCore Gateway MCP target versus a Bedrock action group

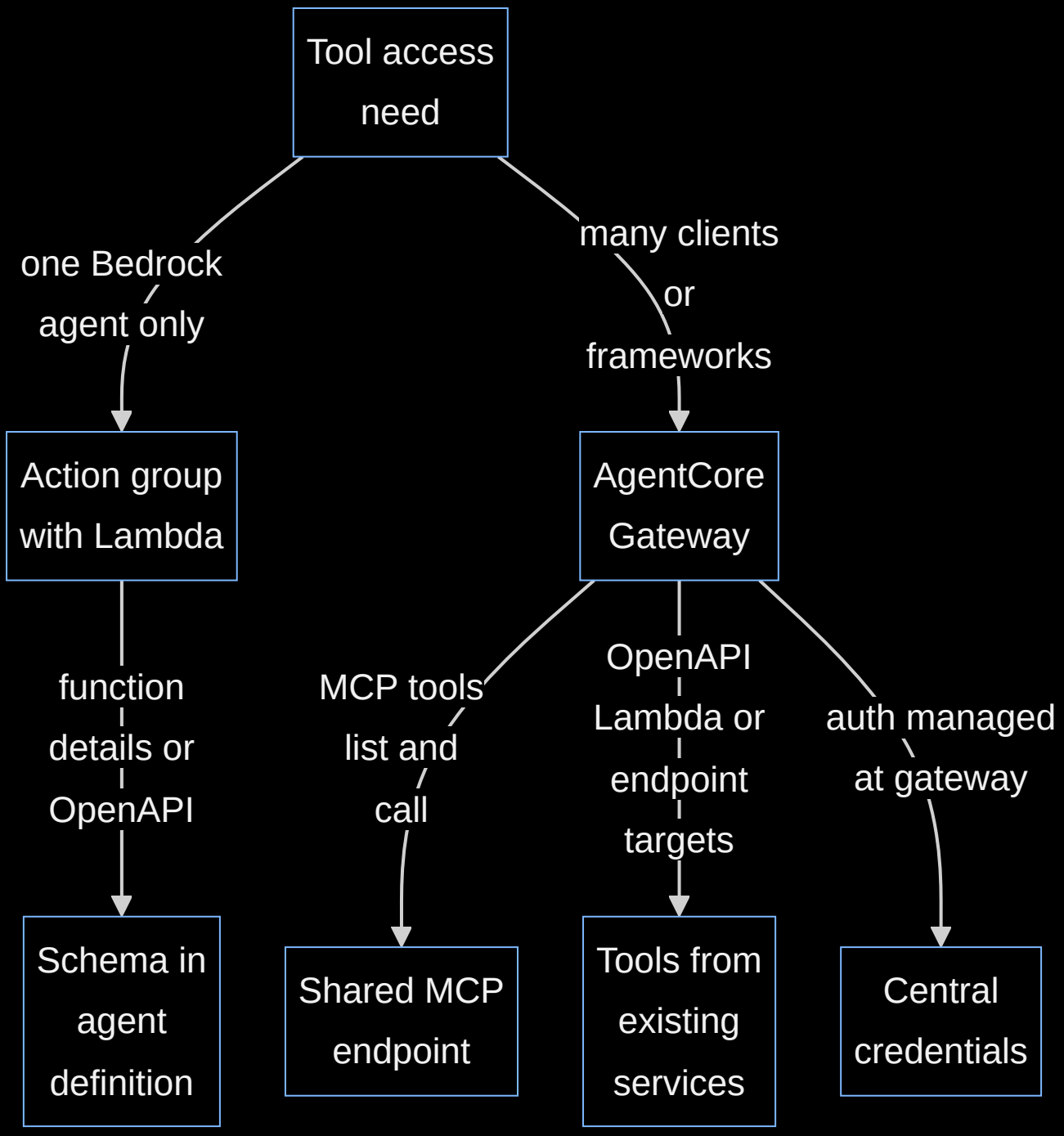


Figure: An action group serves one agent. A gateway serves many clients through one MCP surface.

- **Purpose:** An action group gives one Bedrock agent its tools. Gateway gives any MCP-capable client the same tools.
- **Input:** An action group takes a function or OpenAPI definition with an executor. Gateway takes targets such as OpenAPI APIs, Lambda functions, and existing MCP servers.
- **Output or effect:** An action group returns results only to its agent. Gateway returns tool results to any client that speaks MCP.

Exam tip: The words "one standard tool interface for several frameworks" point to Gateway. The words "our Bedrock agent calls this API" point to an action group.

Rule: Use Gateway when several clients or frameworks need one MCP tool surface. Use an action group for one Bedrock agent.

Trap: Assuming an action group gives MCP access for other frameworks. It is a Bedrock agent mechanism, so other clients need their own adapter.

Source: Sundog course extract, slide page 246 (Gateway targets and MCP conversion) and slide page 256 (MCP as a shared interface). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Gateway description). Target-type behavior details: Not in source in the pages fetched.

Pair 7: Supervisor collaboration versus agents as tools versus Agent Squad

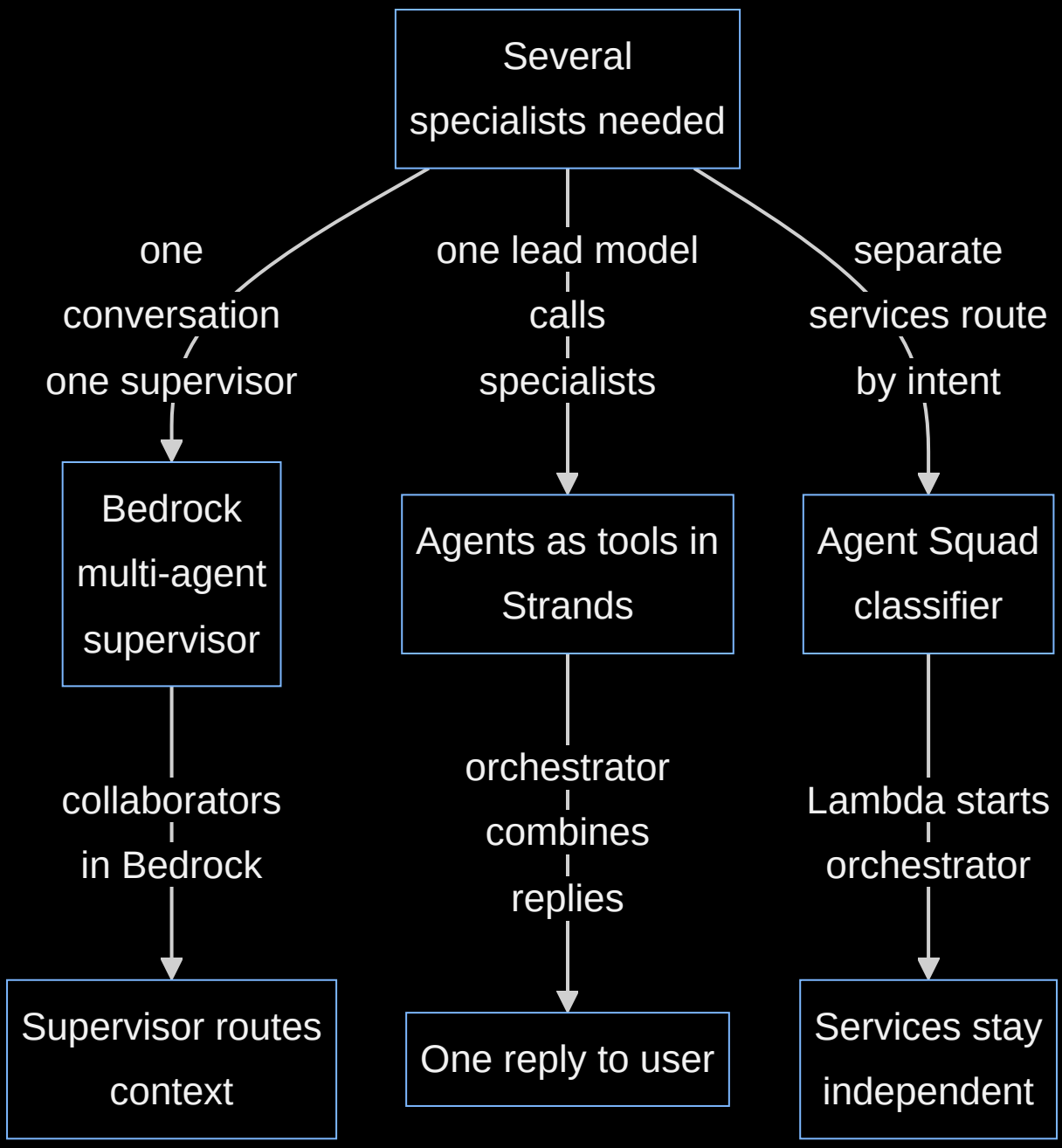


Figure: The three patterns differ in who routes, who combines, and whether services stay separate.

- **Purpose:** A Bedrock supervisor routes one conversation to collaborator agents. Agents as tools lets a lead agent call specialists as functions. Agent Squad routes a request to one of several independent agents.
- **Input:** The supervisor takes a user request and collaborator descriptions. The lead agent takes the request and a list of specialist tools. Agent Squad takes a request and a classifier with agent descriptions.
- **Output or effect:** The supervisor returns a combined answer through its own plan. The lead agent combines specialist results into one reply. Agent Squad

returns the reply of the chosen agent.

Exam tip: "Each specialist stays an independent service" points to Agent Squad. "One reply combines several specialists" points to agents as tools or the supervisor.

Rule: Use one supervisor for a shared conversation. Use agents as tools for one reply. Use Agent Squad for independent services.

Trap: Using one Strands agent with every tool to avoid coordination. Tool choice gets worse as the tool set grows, and the specialists lose focus.

Source: https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html (supervisor and collaborators). Source: Sundog course extract, slide pages 228, 229, 235, 236, and 239 (orchestrator and worker, Strands, Agent Squad).

Pair 8: Session state versus long-term memory versus reference documents

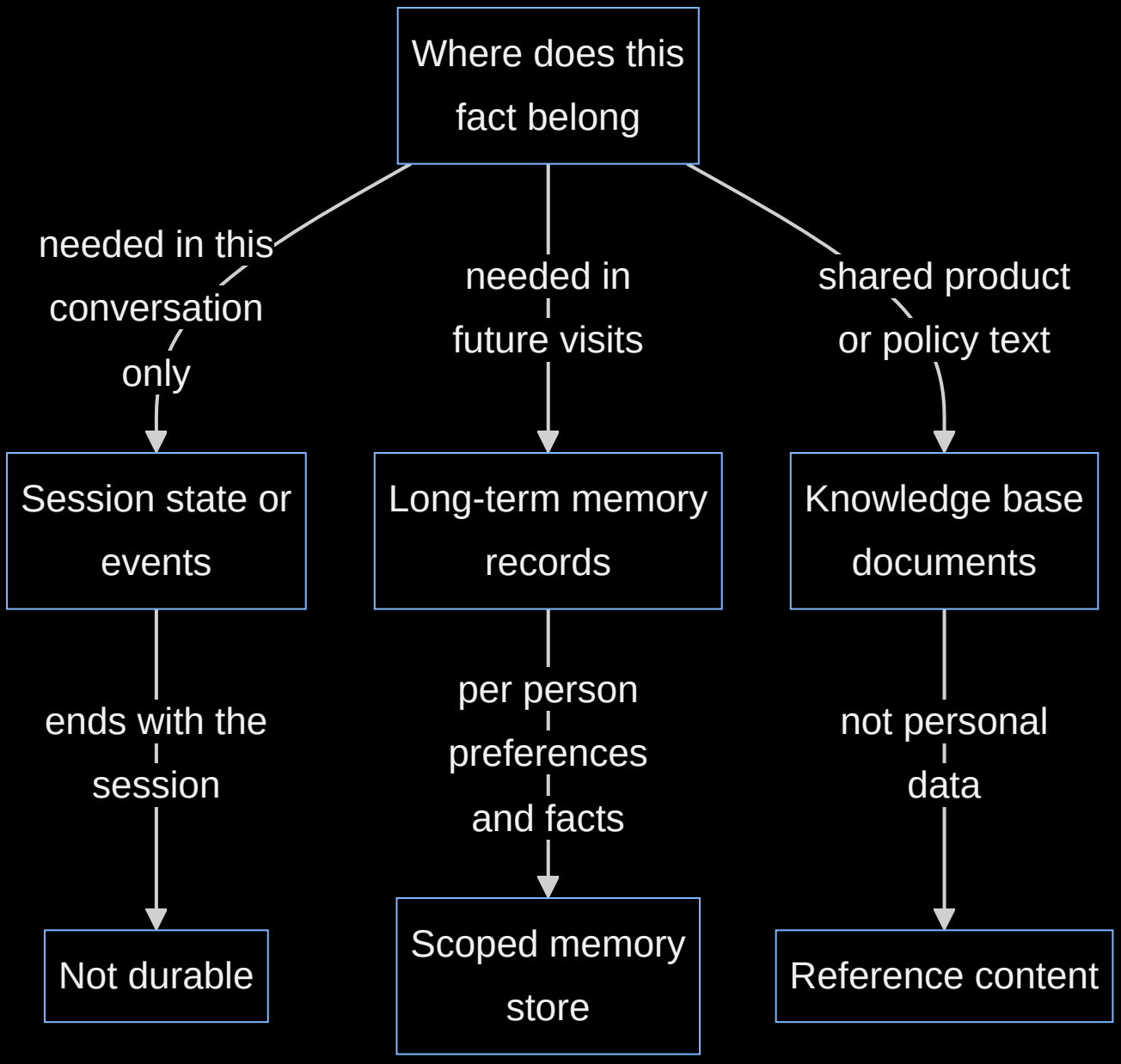


Figure: The lifespan and owner of a fact decide where it is stored.

- **Purpose:** Session state keeps the active thread. Long-term memory keeps person-level facts across sessions. A knowledge base keeps reference content for retrieval.
- **Input:** Session state takes events from the current session. Long-term memory takes extracted insights from many sessions. A knowledge base takes documents you load.
- **Output or effect:** Session state gives the next turn its context. Long-term memory returns recalled records for a person. A knowledge base returns passages for a query.

Exam tip: The words "remember across visits" or "returns next month" point to long-term memory. The words "current troubleshooting thread" point to session state.

Rule: Match each fact to its lifespan. Session for this turn, memory for the person, knowledge base for shared text.

Trap: Storing the preferences of a person in a knowledge base and calling it memory. Reference retrieval does not keep a per-person record that the agent updates.

Source: Sundog course extract, slide page 234 (short-term and long-term memory, knowledge bases framed as a memory type) and slide page 243 (AgentCore Memory). Source: https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html (Memory description).

Read a long question: three worked scenarios

Each scenario below is written for practice. The underlined words are the constraints that decide the design. For each wrong option, the reason names the constraint it breaks.

Scenario 1: approval that waits several days

A bakery group runs a chat helper that places supplier orders. An order can go out only after a regional manager approves it through a web form, and approvals can take **three days**. The design must **not poll** the approval store, and the form posts to an endpoint in **another AWS account**.

- Constraint words: three days, must not poll, another AWS account.
- Express workflow with a callback: fails. Express does not support the callback pattern, and its duration is five minutes.
- Scheduled Lambda that checks a table: fails. It polls, which the design forbids.
- Passing the task token to the other account so its service calls the callback API directly: fails. Callback tokens work only from the same account as the workflow.

- Bedrock Agents that holds the order open: fails. A managed agent loop does not own a multi-day external wait.

Rule: Answer. A Standard workflow with a callback task. The approval endpoint in the other account posts to an API in the workflow account, and a same-account Lambda function sends SendTaskSuccess or SendTaskFailure.

Scenario 2: fixed chain for staff and memory across visits

A dental clinic runs a booking assistant for patients. The assistant must **remember the preferred hygienist of each patient across visits**, and a nightly summary must **always run the same four steps in order**. Clinic staff must **change the step prompts visually without a developer**.

- Constraint words: across visits, always the same four steps, visually without a developer.
- Bedrock Agents session state only: fails. Session state ends with the session, so the preference does not return next visit.
- Step Functions with coded prompts: fails. Staff cannot change the prompts without a developer.
- An AgentCore agent that picks the four steps itself: fails. The steps must be fixed, and the model would choose them.
- Bedrock Flows for the nightly chain with AgentCore Memory for the preference: fits all three constraints.

Rule: Answer. Bedrock Flows for the fixed nightly chain, and AgentCore Memory with a per-patient long-term record for the preferred hygienist.

Scenario 3: many tool vendors and a refusal rule

A port authority runs a container-tracking agent. Three vendors supply tools, and each vendor exposes an MCP server that runs on a laptop during development and as an HTTP service in production. Auditors require the agent to **refuse any tool call that moves a container out of a customs hold**, even if the model asks for it.

- Constraint words: three vendors, MCP laptop and HTTP production, refuse any tool call.
- One Bedrock action group per vendor: fails. It duplicates integration and does not give one shared MCP surface for other frameworks.
- A rule in the system prompt only: fails. The prompt is advice to the model, and the model can ignore it.
- A single Lambda that holds all three vendors and calls them in order: fails. It fixes the order, which the vendors do not need, and it adds a custom router.
- AgentCore Gateway with MCP targets and AgentCore Policy on each tool call, with stdio for development: fits all three constraints.

Rule: Answer. AgentCore Gateway with MCP targets, AgentCore Policy to block the forbidden call before it runs, and stdio for local development.

Distractor patterns seen in the tests

Wrong options in this area follow a few concept-level patterns. Learn the pattern, and you can reject an option even when you do not know every service detail. Each pattern below names the reason it fails.

- **Right service for a sub-part.** An option names a real service that handles one step, such as Lambda, but the question needs the whole orchestration. Lambda runs code and does not coordinate waits.
- **Agent for fixed steps.** The option gives a model the choice of steps for a process with a fixed order. The result adds variation and cost that the rules did not ask for.
- **Workflow for open choices.** The option draws every branch in advance for a task where the next step depends on new results. The workflow cannot handle paths nobody drew.
- **Prompt as a control.** The option stops a behavior by writing an instruction. A prompt is advice, so a rule that must hold needs IAM, Policy, or code.
- **Express for long work.** The option uses Express for an approval or a long wait. Express stops at five minutes and does not support callbacks.

- **Polling for a reply.** The option checks a table on a schedule to see whether a human answered. The question usually forbids polling, and a callback is the native pattern.
- **Token across accounts.** The option sends a callback token to another account that calls the callback API. The token is accepted only from the same account.
- **Logs for reasoning.** The option uses an audit log or model invocation log to explain a wrong tool choice. Those show calls, but the agent trace shows the path.
- **Memory as a bigger prompt.** The option stores every transcript and sends it back each turn. Cost and context grow with each turn, and no retention rule exists.
- **Timeout as a loop fix.** The option raises a timeout or memory to stop a repeating loop. The loop continues and the cost grows, so a loop limit and a fail path are needed.
- **Retry without limits.** The option retries the same failing call with no jitter and no fallback. A shared outage then gets worse.

Trap: Accepting an option only because it names the right service. Check whether it also meets the wait, the owner of each step, and the rule that must hold.

Drill

Each prompt has one answer on the next line. Cover the answer and try first.

1. A model proposes a tool call, and your service must run it and send the result back. Which pattern fits?

Rule: Converse tool use, where your code runs the tool and returns a toolResult.

2. Product analysts reorder prompt steps and add a condition in a visual builder. Which service fits?

Rule: Bedrock Flows with prompt nodes and conditional connections.

3. A reviewer may reply after five days, and no service job runs in the meantime. Which Step Functions type and pattern fit?

Rule: Standard workflow with the wait for a task token, and a heartbeat interval.

4. A payment step must never run twice when a retry happens. Which workflow type fits?

Rule: Standard workflow, because it follows the exactly-once model.

5. Forty thousand files each need their own model call and a retry of only the failed files. Which Step Functions feature fits?

Rule: Distributed Map in a Standard workflow, with one child run per item.

6. A Lambda function must run a job that takes about twenty minutes. What is the design fix?

Rule: Split the job into steps under Step Functions. A single Lambda timeout cannot exceed fifteen minutes.

7. A Bedrock agent must ask for missing values and run three named functions on one Lambda. Which action group style fits?

Rule: Function details, which name each function and its parameters.

8. An auditor must see which knowledge base and which action ran for one answer. Which feature gives that?

Rule: Agent trace, requested with the trace flag on the InvokeAgent call.

9. Several agent frameworks must call the same retrieval tools through one standard interface. Which AgentCore service fits?

Rule: AgentCore Gateway with MCP targets for the shared tools.

10. A tool call that moves restricted cargo must be refused, even when the model requests it. Which control fits?

Rule: AgentCore Policy, which evaluates each tool call outside the agent code.

11. The assistant of a clinic must recall the preferences of each patient at the next visit. Which store fits?

Rule: AgentCore Memory long-term records, scoped to the person.

12. Independent microservices each handle one domain, and a classifier must route each request to one of them. Which framework fits?

Rule: AWS Agent Squad with a classifier, because it routes between separate agents.

Sources

Source: Each URL below was fetched with WebFetch. The first list shows pages that returned usable text. The second list shows pages that returned only a title or an index, and no fact in this module depends on them.

- https://docs.aws.amazon.com/bedrock/latest/userguide/agents-action-create.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/agents-returncontrol.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/agents-multi-agent-collaboration.html
- https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html
- https://docs.aws.amazon.com/bedrock/latest/userguide/flows.html
- https://docs.aws.amazon.com/step-functions/latest/dg/connect-bedrock.html
- https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html
- https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html
- https://docs.aws.amazon.com/lambda/latest/dg/configuration-timeout.html

- https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how-it-works.html (title only, no facts used)
- https://docs.aws.amazon.com/bedrock/latest/userguide/flows-how-it-works.html (topic index only, no facts used)

Source: Sundog course extract for this module, file m4-agents-orchestration-sundog.txt, slide pages cited in each section.

All modules Previous Next